# AO 对接配合清单(给付卓新)

# AO 对接配合清单(给测试侧 · 付卓新)

> 回应《AO Neo4j 接入对齐讨论与信息请求》。下面每一条都是 **2026-06-23 在 AO 机器上实测**(真发了 curl / 读了代码)后给的结论,不是文档推断。 一句话定调:**不用再搭新网络;方案直接走"路线 B(HTTP :3500)";只有 1 个必须改的点(IP)+ 2 个会绊到你的坑(测试 id 失效、关系不是图边)。**

---

## 0. 先回答你最关心的两件事

**网络**:不需要共同连新网络、不需要 VPN/堡垒机。你**已经能打通 :3002 执行 API**,而 :3500 本体 API 和它**是同一台机器、同一网段、只是换端口**——`:3500` 绑在 `*:3500`(所有网卡,实测),所以你能到 :3002 就一定能到 :3500。

**唯一必须改**:旧接入卡写的 `192.168.1.104` **已失效**(ping 100% 丢包,根本不是本机网卡)。AO 这台机器现在是 `192.168.1.111`。把你 env 里 ontology / execution 两个 base URL 的 IP 都改成 `.111`。

> AO 侧会把这台机器固定成静态 IP,免得 DHCP 再漂(这就是 `.104` 变 `.111` 的根因)。

**方案**:路线 A(bolt 直连)整条放弃。原因:**AO 自己没有独立 Neo4j**,AO 真 Agent 读本体本来就**只走 :3500 这个 HTTP API**。所以你走 :3500(路线 B)读到的,就是真 Agent 跑用例时看到的同一份图——这正是你要的"单一数据源 + 真实可比对",且零新增基建。bolt 那层的 Neo4j 属于陈洋的 Allmeta Studio,AO 没法单方面开,而且走它反而和真 Agent 的取数路径不一致。

---

## 1. 连接参数(实测可用)

| 服务 | Base URL | 鉴权 header |
| --- | --- | --- |
| 本体 API(:3500,Allmeta | `http://192.168.1.111:3500` | `Authorization: Bearer abc123` |

| 服务 | Base URL | 鉴权 header |
| --- | --- | --- |
| Studio,背靠 Neo4j) | | |
| **执行 API**(:3002) | `http://192.168.1.111:3002/api/agent-execution` | `Authorization: Bearer aek_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30` |

- **两把 token 不通用**(实测):`abc123` 发到 :3002 → 401;`aek_live_...` 发到 :3500 → 401。各用各的。
- **domain 含中文,必须 percent-encode**:curl 用 `-G --data-urlencode "domain=招聘-v1"`。`RAAS-v1` / `raas` 是历史别名,统一以 `招聘-v1` 为准。
- 你 env 直接这么填:

```
AO_ONTOLOGY_BASE_URL = http://192.168.1.111:3500
ALLMETA_API_KEY      = abc123
AGENT_EXECUTION_BASE = http://192.168.1.111:3002/api/agent-execution
AGENT_EXECUTION_API_KEY = aek_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30
AO_ONTOLOGY_VERSION  = rules_v0_1_003   # 见 §5 的重要说明
```

> ⚠️ `abc123` 是个很弱的开发 token(归陈洋的 Studio `.env` 管)。AO 侧已在排期轮换;轮换后会连同静态 IP 一起给你一版干净接入卡。

---

## 2. 七个本体端点 —— 全部实测 200(附可直接粘的 curl)

域全部用 `--data-urlencode "domain=招聘-v1"`,token `-H "Authorization: Bearer abc123"`,下面省略。

| # | 端点 | 实测 | 返回形状 / 要点 |
| --- | --- | --- | --- |
| 1 | `GET /api/v1/ontology/rules?limit=1000` | 200 | `{items:[Rule], nextCursor}`。**全集条**,`limit=1000` 一页拉完(`nextCursor` |

| # | 端点 | 实测 | 返回形状 / 要点 |
|---|---|---|---|
| | | | 就是你的覆盖率分母来源 |
| 2 | `GET /api/v1/ontology/rules/{id}`(如 `10-5`) | 200 | 单个 Rule(无 items 包裹)。`standardizedLogicRule` = 你做 `ruleTextSnapshot.sha256` 的那段原文 |
| 3 | `GET /api/v1/ontology/actions/ruleCheckForMatchResume/rules` | 200 | 比名字更全:整个 Action + `action_ste` `rules[]` + `ruleCount` + `userPromp` **Action→Step→Rule 链** |
| 4 | `GET /api/v1/ontology/instances/{Label}/{id}` | 200 | 单实例。**注意 id 格式见 §4**(旧卡的 `C-2` `001` 已失效) |
| 5 | `GET /api/v1/ontology/instances/{Label}?<filter>=` | 200 | `{items, nextCursor}`。**任意属性都能** `&candidate_id=C_5eacb369` 实测精确 |
| 6 | `GET /api/v1/ontology/links?from=<id>&type=<REL>` | 200 | `{items, nextCursor}`。**当前 domain** **`items:[]`** —— 见 §6,关系不是图边 |
| 7 | `GET /api/v1/ontology/schema/rules` | 200 | `{label,resource,sampledNodes:100` `[{name,occurrence}]}`,15 个字段 |

**翻页**:列表端点(rules/instances/links)的 `nextCursor` 是 `{"skip":N}` 的 base64(`eyJza2lwIjozfQ` = `{"skip":3}`)。非 null 时回传 `&cursor=<nextCursor>` 取下一页,null 即末页。单对象端点(单规则/单实例/action 链/schema)没有 cursor。

**错误形状**(都是干净 JSON,字段 `error` + `message`): `401 unauthorized`(token 缺/错)· `400 missing-domain` · `404 instance-not-found` / `node-not-found` · `502 neo4j-unavailable`。

**数据小坑**:Neo4j 时间字段(`updatedAt` 等)序列化成嵌套 `{year:{low,high},month:{low,high},...}`,读 `.low`,不是 ISO 串(同一节点里 `snapshot_at` 这种又是普通 ISO 串,混着来)。

```
# 1 全集(覆盖率分母):实测 313 条一页拉完
curl -s -m10 -G "http://192.168.1.111:3500/api/v1/ontology/rules" \
  --data-urlencode "domain=招聘-v1" --data-urlencode "limit=1000" \
  -H "Authorization: Bearer abc123"

# 3 Action→Step→Rule 链(对账 retrievedRuleIds 用)
curl -s -m10 -G "http://192.168.1.111:3500/api/v1/ontology/actions/ruleCheckForMatchResume/rules" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer abc123"
```

## 3. 逐条回应你的 5 个请求(①–⑤)

### ① 完整端点手册 —— ✅ 给你

随这份一起发你:`docs/action_object_prompt/ONTOLOGY-API-USER-GUIDE-BASED-ON-NEO4J.md`(1427 行,含全部端点 + 请求参数 + 响应 schema + 翻页约定)。你之前只收到接入卡,没收到这份。

### ② 两个端口的 token —— ✅ 不是同一把

- :3002 执行 API = `aek_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30`(你已生效)
- :3500 本体 API = **`abc123`**(新给你的,见 §1)
- 实测互相串用都会 401。两套独立鉴权域。

### ③ 用例 id 存在性核对 —— ⚠️ 两个都已失效,给你替身

实测(本体 Neo4j):

- `Candidate / C-20260424-001` → 404 instance-not-found
- `Job_Requisition / R2026031629581` → 404 instance-not-found

**已验证可用的替身 id:** | Label | 替身 id | 说明 | |---|---|---| | Candidate | **`C_5eacb369`** | 周七 / Zhou Qi(单查 200) | | Job_Requisition | **`mock_jr_v0_1_010_1778766302725`** | 商务 BD 北京(单查 200) |

- candidate id 真实格式是 `C_<hex>` / `mock_cand_*` / `test_ping`,**不是** `C-YYYYMMDD-NNN`;节点上 id 字段名是 `candidate_id`(不是 `id`),岗位是 `job_requisition_id`。
- 当前 `招聘-v1` 的实例都是 `mock_* / smoke_* / *_TEST* / e2e_*` 这类会被重置的数据,**没有长期稳定的固定 fixture**。AO 侧 action item:种一组永不删的标准测试三元组(Candidate+Job_Requisition+Application),给你一个能写死的 id。在那之前请用上面的替身,并做好 id 可能再变的准备。
- 提醒:**这两个 id 在执行 API(:3002)那边是按 store ref 读的(可能走 partner-pg,不一定是本体 Neo4j)**。我这次只确认了它们在**本体 Neo4j** 里没了;执行侧的验收用例(指南 §10 tc-de5ec703)我没实跑,你那边要用请单独确认 store 里还在不在。

### ④ 域名规范 —— ✅ 统一用 `招聘-v1`

`RAAS-v1` / `raas` 都解析到它,但请你代码里统一切到 `招聘-v1` 为权威名。

### ⑤ 版本 pin —— ⚠️ 有个重要的"指纹≠执行"区别,务必看

- 可 pin 的版本只有两个(实测 `/capabilities`):`rules_v0_1_002`(248 条,meta 0.2)、`rules_v0_1_003`(261 条,meta 0.3)。

- **执行请求里 `ontology.version` 是必填**,AO **没有默认值**;填空/填未知值会硬报 `ONTOLOGY_VERSION_UNAVAILABLE`(不可重试)。**请显式填 `rules_v0_1_003`**(最新)。
- **关键区别(这条直接影响你的覆盖率分母)**:目前的 version pin 只用于**校验 + 指纹,真正的 matchResume 判定是跑在 LIVE Neo4j 上的——当前 LIVE 有 313 条规则,比 261 的快照多 52 条**。也就是说 pin `rules_v0_1_003` 并不代表引擎只评 261 条;它评的是 313 条的实时图。AO 在响应里用 `meta.ontologyLoaded.liveConsistency` 暴露这个差异。
- **所以**:
  - 你要对齐"AO 实际评估面"的覆盖率分母 → 用 **:3500 实时全集(313 条,再按 `executor='Agent'` 且 `enforcementLevel='mandatory'` 自筛),不要**用 261 的快照数。
  - 261/248 的快照只是用于指纹复现。
  - :3500 的 `/rules` **不支持 `&version=` 过滤**(实测带上返回 0 条);要拿冻结的 248/261 条原文得用 AO 的 snapshot 文件,不能从 live API 取。
  - AO 侧 action item:把 live(313)和最新快照(261)对齐(重生成 `rules_v0_1_004` 或修剪 live),消除这个 52 条的缺口。

---

## 4. 你那 5 类图遍历 Cypher 的 HTTP 覆盖结论

读完整手册 + 实测后,**本体 API 是"Neo4j 之上的 CRUD",不是图遍历层**(手册"What's not in v1"明说:无 Workflow CRUD、无裸 Cypher、无路径接口)。逐条:

| # | 你的 Cypher | 结论 | 替代 / 说明 |
|---|---|---|---|
| 1 | `shortestPath(...)` | **无对应 (GAP)** | 没有 path/shortestPath/graph 端点(实测全 404) |
| 2 | `[:CHECKS\|GOVERNS*0..]` 规则→下游消费者 | **部分(只反向)** | 只有 action 视角: `GET /actions/{ref}/rules` 内部走 Action→Step→Rule。要"给定规则查谁用它",得枚举 `/actions` 再逐个调 `/rules` 反查;没有 `/rules/{id}/consumers` |
| 3 | `(:Workflow)-[:RUNS_ACTION]->(:Action)` | **无对应 (GAP)** | 根本没有 workflow 端点( `/workflows` →404),Workflow 在本 API 里不是建模实体 |
| 4 | `(:Client)-[*1..2]-(:Job_Requisition)` | **无对应 (GAP)** | 没有变长遍历。只能**客户端按 FK 字段自己 join**(见 §6);且当前 `招聘-v1` 的 `Client` 实例为 0 行,连锚点都没有 |
| 5 | `type(rel)` 关系类型发现 | **无对应 (GAP)** | `/schema` 只做节点属性自省; `/links?type=` 一次只能验一个类型 |

**你漏掉的 bonus 端点(建议补进客户端):** `GET /objects` `/actions` `/events`(四类节点通用 CRUD,枚举 Action 给 #2 用)· `GET /schema`(一次拿四类 label,不只 `/schema/rules`)· `GET /actions/{ref}/steps`(只要步骤不要规则)· 完整 `/links` 的 POST/PUT/PATCH/DELETE · `POST /actions/matchResume/results`(单 action 写结果)。这些都**不**增加路径/变长遍历能力。

---

## 5. 关系不是图边 —— 这条会改变你的实现(重要)

实测: `/links` 对**当前 domain 里所有节点**(失效候选人、两个活候选人、活岗位)都返回 `items:[]`;不带 `from` 也是空。也就是说 `招聘-v1` **里的业务关系不是存成 Neo4j 边,而是存成节点上的外键字段**——例如 `Application` 节点带 `candidate_id` + `job_requisition_id`。

对你的影响:

- 候选人↔岗位的遍历(你 L3 grounding 那块),**不要等图边**,改成:读 `Application` / 节点实例的 prop bag,在你自己代码里按 `candidate_id` / `job_requisition_id` 等 FK 属性 join。
- `EMPLOYED_BY` 这类 `/links` 调用现在拿不到数据,别依赖。
- AO 侧 action item:确认 `招聘-v1` 到底是"边没灌"还是"模型本就用 FK 不用边",并在接入卡里写清,免得你按图遍历预期去对接。

---

## 6. 执行 API(:3002)对接契约(你已接通,这里补权威细节)

- **异步 shadow 模式**: `POST /executions` → 202 + `executionId` → 轮询 `GET /executions/{id}` 到 `status=succeeded/failed`(2–5s 一次,单用例 ≤3 分钟)→ 需要全文再 `GET /executions/{id}/llm-calls/{callId}`。`shadow` 是唯一模式(判定真跑,副作用全抑制并在 `result.emittedEvents` / `trace.steps[].suppressedSideEffects` 申报)。
- **liveness 探针 =** `GET /capabilities`(返回 JSON)。**别**拿裸 `GET /api/agent-execution` 当健康检查(返回的是 Next 应用 HTML,不是 JSON);没有 `/health`、`/ping`。
- **join key —— 你最关心的对账**: `trace.retrievedRuleIds[]` 和 `ruleEvaluations[].ruleId` 就是**本体原始 id(如 `10-5`),端到端零改写**,和 :3500 `/rules` 返回的 id 完全一致 → **直接字符串等值 join,安全**。
- **并发**: `maxConcurrent=4`(实测)。超了 → `429` + `Retry-After`,body `{error.code:RATE_LIMITED, retryable:true}`,按 `Retry-After` 退避。
- **幂等**: `clientRequestId` 是必填幂等键,同 id 永远返回同 `executionId`、不重跑(≥7 天);结果保留 7 天。
- **覆盖率口径对齐**:AO 的评估面只含 `executor='Agent'` 且 `enforcementLevel='mandatory'` 的规则——你算覆盖率时用同样的筛选口径。

- **两个状态语义差异**(和你 5 态模型不同,别合并):`ruleEvaluations[].status` 有第 6 态 `evaluated_inconclusive`(它才是 `decision=needs_review` 的来源);`retrieved_not_evaluated` 在 AO 指"被确定性预筛排除、从未进 LLM 上下文"(单 pass),和你"进了上下文但没走到"的定义不一样。

> ⚠️ 稳定性:实测第一发 curl 撞到一次 connection-refused(Next dev server 正在热重载),随后连续 200。你**大批量发用例(30–60 条)前**,请 AO 确认 :3002 是用 `npm run start`(常驻)而不是热重载的 dev server 起的,否则中途重载会变成零星 connection-refused(不是能干净重试的 4xx/5xx)。

---

## 7. AO 侧待办(给 AO 团队,不用你跟进)

1. 把这台机器**固定静态 IP**(现 `.111` 是 DHCP)—— 根治 base URL 漂移。
2. 在 `招聘-v1` **种一组永不删的标准测试三元组**(Candidate+Job_Requisition+Application),给测试侧一个能写死的 id;同步更新接入卡 / 指南 §10 里失效的 `C-20260424-001` / `R2026031629581`。(接入卡里的 IP+示例 id 已于 2026-06-23 先行更正。)
3. **对齐 live(313)与最新快照(261)**:重生成 `rules_v0_1_004` 或修剪 live;并在执行响应里把 `liveConsistency`(如 live 313 vs pinned 261)落成具体数字。
4. 决定 version pin 的真实语义:要么实现"按版本加载规则",要么明确标注"版本=指纹/校验,判定跑 live"。
5. 跟陈洋一起**轮换 :3500 的弱 token** `abc123`;轮换后连静态 IP 出一版干净接入卡。 6.(可选)评估 5 个图遍历 GAP 是否要补端
   点:`/path`(shortestPath)、`/workflows` + `/workflows/{ref}/actions`、`/instances/{label}/{value}/neighbors?hops=1..2`、`/schema/relationships`、`/rules/{id}/consumers`。
6. 确认 :3500/:3002 长期是否要全网卡暴露,还是限到 LAN 子网/partner host。

# AO Neo4j 本体 API 接入卡

# AO Neo4j 本体 API — 接入卡(给付卓新)

> 这是 AO 的 **Neo4j 本体只读接口**(Allmeta Ontology API,直接背靠 Neo4j)。 你用它做:① 覆盖率分母(域内规则全集)② 核对规则原文/版本指纹 ③ 归因第 5 步核对候选人/岗位权威数据。 完整 endpoint 手册见同目录:ONTOLOGY-API-USER-GUIDE-BASED-ON-NEO4J.md(本卡只给你联调最常用的几条 + 真实连接参数)。

---

## 1. 连接参数

| 项 | 值 |
|---|---|
| Base URL | `http://192.168.1.111:3500`(2026-06-23 实测纠正:旧卡写的 `.104` 已失效/不可达,现为 `.111`) |
| 鉴权 | `Authorization: Bearer abc123` 或 `x-api-key: abc123`(:3500 本体 API 的 token,**与 :3002 执行 API 的 `aek_live_...` 不是同一把**) |
| 域 (domain) | `招聘-v1`(请求都要带 `?domain=招聘-v1`;`RAAS-v1` / `raas` 是历史别名,以 `招聘-v1` 为准) |

> ⚠️ domain 含中文,URL 里要 percent-encode(curl 用 `--data-urlencode "domain=招聘-v1"`)。

---

## 2. 你联调最常用的 4 条

### ① 域内规则全集 —— 覆盖率分母的正确来源

```
curl -s -G "http://192.168.1.111:3500/api/v1/ontology/rules" \
  --data-urlencode "domain=招聘-v1" --data-urlencode "limit=1000" \
```

```
  -H "Authorization: Bearer $TOK"
# → { "items": [ {id, businessLogicRuleName, standardizedLogicRule, executor, enforcementLevel,
        failurePolicy, applicableClient, applicableDepartment, ...}, ... ] }
```

- 这是**不预过滤**的全集(含 `executor=Human` / `optional`)。算 Agent 覆盖率分母时,按 `executor='Agent'` 且 `enforcementLevel='mandatory'` 自行筛一遍(这是 AO 执行引擎的评估口径)。
- 支持 `?cursor=` 翻页;`?limit` 默认 100、最大 1000。

### ② 执行引擎同款规则面(matchResume 的 Action 规则,按步骤分组)

```
curl -s -G "http://192.168.1.111:3500/api/v1/ontology/actions/ruleCheckForMatchResume/rules" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer $TOK"
# → { id, name, action_steps:[{id, order, name, rules:[...]}], rules:[...扁平去重...], ruleCount }
```

这是执行 API 评估时实际拉取规则的同一接口,适合和执行 API 返回的 `retrievedRuleIds` 对账。

### ③ 单条规则原文(核对 ruleTextSnapshot)

```
curl -s -G "http://192.168.1.111:3500/api/v1/ontology/rules/10-5" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer $TOK"
```

`standardizedLogicRule` 就是执行 API `ruleEvaluations[].ruleTextSnapshot.sha256` 取哈希的那段全文。

### ④ 候选人/岗位实例读取(归因第 5 步核对权威数据)

```
curl -s -G "http://192.168.1.111:3500/api/v1/ontology/instances/Candidate/C_5eacb369" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer $TOK"
curl -s -G
        "http://192.168.1.111:3500/api/v1/ontology/instances/Job_Requisition/mock_jr_v0_1_010_177876630272
        5" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer $TOK"
# 列表查询(如某候选人名下简历):/api/v1/ontology/instances/Resume?domain=招聘-v1&candidate_id=C-...
```

Label 取值:`Candidate` / `Resume` / `Job_Requisition` / `Application` / `Blacklist`;链接查询走 `/api/v1/ontology/links?domain=招聘-v1&from=<id>&type=EMPLOYED_BY`。

---

## 3. 约定 & 错误

- 节点是 property-bag 语义:嵌套对象/数组以 JSON 字符串存储,读出后按需 `JSON.parse`。做 schema 校验请用"允许未知字段"模式。
- 错误形状:`{ "error": "<code>", "message": "..." }`。常见:`401 unauthorized`(token 缺/错)、`400 missing-domain`、`404 node-not-found` / `action-not-found`、`502 neo4j-unavailable`(Neo4j 不可达)。
- 字段全集/schema 自描述:`GET /api/v1/ontology/schema/rules?domain=招聘-v1`。

---

## 4. 与执行 API 的关系

- **执行 API**(`http://192.168.1.111:3002/api/agent-execution`,见《调用指南》):你发用例,AO 用生产引擎跑、回结论+trace。**AO 自己会用本 Neo4j API 取数**,你不必代取。
- **本 Neo4j API**:给你**旁路**核对用——覆盖率分母、规则原文指纹、归因第 5 步的数据核对。两者 domain 一致(`招聘-v1`)、规则 id 一致(`10-5` 这种本体原始 id),可直接 join。

# AO 本体 API 完整端点手册

# Ontology API — User Guide (Neo4j-backed)

A central HTTP API at `/api/v1/ontology/` that exposes CRUD over the ontology graph stored in Neo4j. Hosted by the Studio app (`apps/studio`, port 3500). All five artifact kinds — DataObjects, Rules, Actions, Events, and Links (Neo4j relationships) — share one base URL, one auth scheme, and one error shape.

This guide is for **API consumers** (link-generator, future builder apps, agents, ad-hoc scripts). For the architecture and rationale see `docs/superpowers/specs/2026-04-29-ontology-api-design.md`.

---

## At a glance

| Group | Path prefix | Operations | Notes |
|---|---|---|---|
| DataObjects | `/api/v1/ontology/objects` | C R U D (PUT, PATCH, DELETE) | Node label `:DataObject` |
| Rules | `/api/v1/ontology/rules` | C R U D | Node label `:Rule` |
| Actions | `/api/v1/ontology/actions` | C R U D | Node label `:Action` |
| Events | `/api/v1/ontology/events` | C R U D | Node label `:Event` |
| Links | `/api/v1/ontology/links` | C R U D | Neo4j relationships, identified by `linkId` |
| Schema | `/api/v1/ontology/schema` | R | Live property introspection |
| Action sub-resources | `/api/v1/ontology/actions/{ref}/...` | composite (rules-by-action, steps-by-action, per-action result writers) | `{ref}` matches `Action.id` then `Action.name` |

| Group | Path prefix | Operations | Notes |
|---|---|---|---|
| Instance data | `/api/v1/ontology/instances/{label}` | C R U D | Records typed by a `:DataObject` schema (e.g. `:Candidate {candidate_id, …}`). PK field discovered from the schema; opt-in `?validate=strict` |

- **Base URL** — `http://localhost:3500` in dev. The `/api/v1/ontology` prefix is the API version path; future incompatible changes will land under `/api/v2/ontology`.
- **Content type** — JSON. Every request and response body is `application/json` unless the response is empty (e.g. `204 No Content`).
- **Versioning** — `/api/v1/ontology/...` is stable. Breaking changes ship behind a new path segment, never silently.

---

## Authentication

All endpoints require a shared bearer token. Configure it on the host (Studio) via:

```
ONTOLOGY_API_TOKEN=<your-strong-random-key>
```

Send on every request via **either**:

```
Authorization: Bearer <token>
```

```
x-api-key: <token>
```

Constant-time comparison server-side. Failure modes:

| Condition | HTTP | `error` |
|---|---|---|
| Server has no `ONTOLOGY_API_TOKEN` | 500 | `server-misconfigured` |
| Header missing | 401 | `unauthorized` |
| Token present but wrong | 401 | `unauthorized` |

## Common conventions

### Domain scoping

Every artifact is scoped to a domain (`RAAS-v1`, `R7-001`, …). Every endpoint takes the domain as **either** a `?domain=…` query param (reads, deletes, schema) **or** as a `domainId` field in the JSON body (writes). The server adds `WHERE n.domainId = $domain` to every Cypher match — there is no way to query across domains in v1.

| Operation | Where domain goes |
|---|---|
| `GET /…` | `?domain=RAAS-v1` |
| `GET /…/{id}` | `?domain=RAAS-v1` |
| `DELETE /…/{id}` | `?domain=RAAS-v1` |
| `POST /…` | body: `{"domainId": "RAAS-v1", …}` |
| `PUT /…/{id}` | body: `{"domainId": "RAAS-v1", …}` |
| `PATCH /…/{id}` | body: `{"domainId": "RAAS-v1", …}` |

A request without a domain returns `400 missing-domain`.

### Property-bag semantics

The API is **schema-agnostic**. It does not hardcode field lists for DataObject, Rule, Action, or Event. Whatever properties the client sends are stored on the node verbatim; reads return whatever is currently on the node. This is intentional — adding a new field in any builder requires zero API changes.

Two consequences:

1. **No server-side field validation** beyond `id` (string, required) and `domainId` (string, required). The server does not reject unknown keys.
2. **Discover the actual fields** in use via the schema endpoint (`GET /api/v1/ontology/schema/{resource}?domain=…`).

### The flatten rule (writes only)

Neo4j only stores **primitives and arrays of primitives** as node properties. The API handles this on writes:

- `string` / `number` / `boolean` / `null` → stored as-is
- `string[]` / `number[]` / `boolean[]` → stored as-is
- nested objects (`{ … }`) → JSON-stringified to a single property
- arrays of objects (`[{…}, {…}]`) → JSON-stringified

**Reads do not auto-inflate.** What the server stored is what you get back. If you need the structured form, `JSON.parse()` the relevant property on the client. (A future `?inflate=true` query param may re-hydrate flattened properties; not in v1.)

### Pagination

List endpoints accept `?limit=N` (default 100, max 1000) and `?cursor=…` (opaque token returned by the previous page). Iteration is stable as long as `domainId` is fixed.

### Error shape

```
{
  "error":   "<machine-readable-code>",
  "message": "<human-readable detail>",
  "details": { /* optional context, varies by error */ }
}
```

| HTTP | When | `error` examples |
|---|---|---|
| 400 | Invalid body, missing `id` / `domainId`, malformed query | `missing-domain`, `missing-id`, `invalid-json` |
| 401 | Auth missing / invalid | `unauthorized` |
| 404 | Resource not found | `node-not-found`, `link-not-found` |
| 409 | Logical conflict (e.g. link references missing endpoint) | `endpoint-not-found`, `duplicate-id` |
| 500 | Server misconfigured, unexpected programmer error | `server-misconfigured`, `internal-error` |
| 502 | Neo4j unreachable / driver error | `neo4j-unavailable` |

---

## 1. DataObjects, Rules, Actions, Events (generic node CRUD)

All four resources share an identical surface. Substitute `{resource}` with `objects`, `rules`, `actions`, or `events`.

### `GET /api/v1/ontology/{resource}` — list

```
GET /api/v1/ontology/objects?domain=RAAS-v1&limit=50 HTTP/1.1
Authorization: Bearer <token>
```

Response — `200 OK`:

```
{
  "items": [
    { "id": "Job_Requisition", "domainId": "RAAS-v1", "name": "招聘岗位", … },
    { "id": "Candidate",       "domainId": "RAAS-v1", "name": "候选人",   … }
  ],
  "nextCursor": null
}
```

Optional filters: `?id=…`, `?name=…` (top-level equality only in v1; each becomes `WHERE n.<key> = $<key>` in the underlying Cypher). Use the schema endpoint to discover legal property names.

### `POST /api/v1/ontology/{resource}` — create or bulk upsert

Single:

```
POST /api/v1/ontology/objects HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId": "RAAS-v1",
  "id":       "Job_Requisition",
  "name":     "招聘岗位",
  "description": "…",
  "fields":   [{ "name": "title", "type": "string" }]
}
```

Bulk (preferred for large imports):

```
POST /api/v1/ontology/objects HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId": "RAAS-v1",
  "items": [
    { "id": "A", "name": "…" },
    { "id": "B", "name": "…" }
  ]
}
```

Semantics: `MERGE` on `(id, domainId)`. Every `POST` is idempotent — calling it twice with the same payload produces the same graph state. Properties of an existing node are **fully replaced** by the request body (same as `PUT`). Use `PATCH` for partial updates.

Response — `200 OK`:

```
{ "upserted": ["Job_Requisition"], "count": 1 }
```

Bulk:

```
{ "upserted": ["A", "B"], "count": 2 }
```

The `fields` array (and any other nested structure) is JSON-stringified on storage; the read response will return it as a string.

### `GET /api/v1/ontology/{resource}/{ref}` — read one

```
GET /api/v1/ontology/objects/Job_Requisition?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

`{ref}` is matched first against the node's `id`, then any per-label id-aliases (`Action.action_id` only in v1 — historical legacy from older import pipelines), then the human-readable name field — so for callers that only know an artifact by name, `/{resource}/{name}` works too. The name field is per-resource: `name` for Objects, Actions, and Events; `businessLogicRuleName` for Rules. Id matches always win when both an id and a different artifact's name resolve to the same string; aliases rank between id and name.

Response — `200 OK`:

```
{
  "id":        "Job_Requisition",
  "domainId": "RAAS-v1",
  "name":      "招聘岗位",
  "fields":    "[{\"name\":\"title\",\"type\":\"string\"}]",
  "updatedAt": "2026-04-29T13:42:11.728+08:00"
}
```


`404 node-not-found` if no node matches `(ref, domainId)`.

### `PUT /api/v1/ontology/{resource}/{ref}` — replace

```
PUT /api/v1/ontology/objects/Job_Requisition HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId":    "RAAS-v1",
  "name":        "招聘岗位",
  "description": "Replaced description"
}
```

Replaces **all** properties on the node. The URL `{ref}` is resolved to the node's actual `id` (id-then-name, id wins on ties); the body's `domainId` plus the resolved `id` are merged in automatically. Properties absent from the body are removed.

Response — `200 OK`: the new node.

### `PATCH /api/v1/ontology/{resource}/{ref}` — partial update

```
PATCH /api/v1/ontology/objects/Job_Requisition HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId":    "RAAS-v1",
  "description": "Updated description only"
}
```

Merges the supplied keys; leaves other properties untouched. To explicitly remove a property, set it to `null` in the body.

Response — `200 OK`: the updated node.

### `DELETE /api/v1/ontology/{resource}/{ref}` — delete

```
DELETE /api/v1/ontology/objects/Job_Requisition?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

Uses `DETACH DELETE` — the node and **all relationships touching it** (in either direction) are removed. There is no soft-delete in v1.

Response — `200 OK`:

```
{ "deleted": 1 }
```

`404 node-not-found` if `{ref}` resolves to neither an id nor a name in the requested domain — same as GET. (Earlier revisions returned `200 {deleted: 0}` for missing nodes; that idempotency promise was dropped on 2026-05-08 along with the name-fallback rollout, since returning 404 on a typo'd ref is more useful than silent success. A `200 {deleted: 0}` response is still possible in the rare TOCTOU window where the node disappears between the resolver step and the delete step.)

---

## 2. Schema introspection

Discover what properties currently exist on a label, sampled from live Neo4j. The API does not maintain a static schema — this endpoint **is** the schema.

### `GET /api/v1/ontology/schema?domain=…` — all four labels

```
GET /api/v1/ontology/schema?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

```
{
  "domain": "RAAS-v1",
  "labels": [
    { "label": "DataObject", "resource": "objects", "sampledNodes": 100,
      "properties": [
        { "name": "id",       "occurrence": 100 },
        { "name": "domainId", "occurrence": 100 },
        { "name": "name",     "occurrence": 98 },
        …
      ] },
```

```
    { "label": "Rule",   "resource": "rules",   … },
    { "label": "Action", "resource": "actions", … },
    { "label": "Event",  "resource": "events",  … }
  ]
}
```

### `GET /api/v1/ontology/schema/{resource}?domain=…` — one label

```
GET /api/v1/ontology/schema/rules?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

```
{
  "label":        "Rule",
  "resource":     "rules",
  "domain":       "RAAS-v1",
  "sampledNodes": 100,
  "properties": [
    { "name": "id",                    "occurrence": 100 },
    { "name": "domainId",              "occurrence": 100 },
    { "name": "businessLogicRuleName", "occurrence": 100 },
    { "name": "specificScenarioStage", "occurrence": 100 },
    { "name": "applicableClient",      "occurrence":  87 },
    …
  ]
}
```

Sampling: the first 100 nodes (per Cypher's natural ordering — not deterministic, but adequate for "which fields are commonly used"). For exact occurrence counts increase the sample with `?sample=N` (max 1000).

---

## 3. Links (Neo4j relationships)

Links are stored as **edges**, not nodes, between existing nodes (typically two `:DataObject`s, but any-to-any is supported). Each link is identified by a `linkId` property the server stores on the relationship; if a client doesn't supply one at create time, the server generates a UUID v4.

### `GET /api/v1/ontology/links` — list

```
GET /api/v1/ontology/links?domain=RAAS-v1&type=HAS_FIELD HTTP/1.1
Authorization: Bearer <token>
```

Optional filters: `?type=…`, `?from=<sourceId>`, `?to=<targetId>`.

```
{
  "items": [
    {
      "linkId":   "f3d1c8e2-…",
      "type":     "HAS_FIELD",
      "fromId":   "Job_Requisition",
      "fromLabel":"DataObject",
      "toId":     "field_title",
      "toLabel":  "DataObject",
      "domainId": "RAAS-v1",
      "confidence": 0.92,
      "updatedAt": "2026-04-29T13:42:11.728+08:00"
    }
  ],
  "nextCursor": null
}
```

### `POST /api/v1/ontology/links` — create

```
POST /api/v1/ontology/links HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId":   "RAAS-v1",
  "type":       "HAS_FIELD",
  "fromId":     "Job_Requisition",
  "toId":       "field_title",
  "confidence": 0.92,
  "linkId":     "optional-client-supplied-id"
}
```

Constraints:

- `type` must match `^[A-Z][A-Z0-9_]{0,63}$` (Neo4j relationship type rule + an allowlist enforced by the server). Unknown types return `400 invalid-link-type`.
- The endpoint nodes (`fromId`, `toId`) must already exist in the domain. If either is missing the call returns `409 endpoint-not-found` with a `details.missing` array.
- `linkId` is optional; when omitted the server generates a UUID v4.

Response — `200 OK`:

```
{ "linkId": "f3d1c8e2-2a4b-4d8c-9f1d-7e6c5b4a3f2d" }
```


### `GET /api/v1/ontology/links/{linkId}` — read one

```
GET /api/v1/ontology/links/f3d1c8e2-…?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

### `PUT /api/v1/ontology/links/{linkId}` / `PATCH .../{linkId}`

Same semantics as nodes (PUT replaces all properties; PATCH merges). You **cannot** change `type`, `fromId`, or `toId` on an existing link — delete and re-create instead.

### `DELETE /api/v1/ontology/links/{linkId}`

```
DELETE /api/v1/ontology/links/f3d1c8e2-…?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

Response — `200 OK`:


```
{ "deleted": 1 }
```


---

## 4. Action sub-resources

Composite endpoints under `/api/v1/ontology/actions/{ref}/...` that aren't generic CRUD — they assemble multi-node graph shapes in one call:

- two **reads** that traverse `(:Action)-[:HAS_STEP]->(:ActionStep)` (and, for `/rules`, the further `(:Rule)-[:GOVERNS]->(:ActionStep)`) to return either the steps or the steps + rules attached to an Action;
- a **per-action write** that records an instance of an action's outcome (currently only `matchResume`).

All three endpoints accept `{ref}` as either the Action's `id` or its `name` (id wins on ties), and require `?domain=…`. The Action match itself is domain-scoped: an Action created in domain `RAAS-v1` is invisible from `?domain=other-domain`.

## `GET /api/v1/ontology/actions/{ref}/rules`

Returns the Action identified by `{ref}` (within the supplied domain) together with its `(:ActionStep)` children and the rules attached to each step. `{ref}` is matched against `Action.id` first, then `Action.action_id`, then `Action.name` — id wins on ties.

Step discovery is dual-path: linked steps via `(:Action)-[:HAS_STEP]->(:ActionStep)` AND, when no linked steps exist, fallback steps whose `action_id` / `actionId` / `parentActionId` property matches the Action's id (or `action_id` alias). Some import pipelines stamp the parent ref on the step instead of creating the relationship; both shapes are surfaced.

Rules attach to each step via two relationship directions — `(:Rule)-[:GOVERNS]->(:ActionStep)` AND `(:ActionStep)-[:CHECKS]->(:Rule)` — and via a JSON-stringified `step.rules_json` / `step.rulesJson` / `step.rules` property fallback. Ids found only in the JSON fallback are hydrated by a single batched follow-up query against `(:Rule)`.

Steps + rules are scoped to the same `?domain=…`. Step order falls back through `coalesce(s.order, s.index, "0")` to tolerate the inconsistent ordering schemas different import pipelines produce.

Query params:

| Name | Required | Notes |
|---|---|---|
| `domain` | yes | Scopes the Action match, the steps list, and the rules list. |

```
curl -sS \
  -H "Authorization: Bearer $ONTOLOGY_API_TOKEN" \
  "http://localhost:3500/api/v1/ontology/actions/matchResume/rules?domain=RAAS-v1" | jq
```

Response — `200 OK`: a single Action object with nested `action_steps[]` (each step's rules in `step.rules`), a flattened deduped `rules[]` at the top level, a `ruleCount`, and a compiled `userPrompt` suitable for handing directly to an LLM:

```
{
  "id":          "matchResume",
  "name":        "matchResume",
  "description": "...",
  "action_steps": [
    {
      "id":    "step-1",
      "name":  "Hard requirements",
      "order": 1,
      "condition": "...",
```

```
      "rules": [
        {
          "id":                       "10-HARD-DEGREE",
          "businessLogicRuleName":    "...",
          "executor":                 "Agent",
          "applicableClient":         "通用",
          "applicableDepartment":     "Recruitment",
          "businessBackgroundReason": "...",
          "domainId":                 "RAAS-v1",
          "relatedEntities":          ["Candidate", "Job_Requisition"],
          "specificScenarioStage":    "hard-requirements",
          "version":                  "1.0"
        }
      ]
    }
  ],
  "rules": [
    {
      "id":                       "10-HARD-DEGREE",
      "businessLogicRuleName":    "...",
      "executor":                 "Agent",
      "applicableClient":         "通用",
      "applicableDepartment":     "Recruitment",
      "businessBackgroundReason": "...",
      "domainId":                 "RAAS-v1",
      "relatedEntities":          ["Candidate", "Job_Requisition"],
      "specificScenarioStage":    "hard-requirements",
      "version":                  "1.0"
    }
  ],
  "ruleCount": 1,
  "userPrompt": "# Ontology rules for Action matchResume (matchResume)\n\nUse the following ontology rules
       ..."
}
```


Each rule carries a curated subset of the `(:Rule)` node's properties — `id`, `businessLogicRuleName`, `standardizedLogicRule`, `submissionCriteria`, `executor`, `ruleSource`, `applicableClient`, `applicableDepartment`, `businessBackgroundReason`, `domainId`, `relatedEntities`, `specificScenarioStage`, `version`. For the full property catalog stored on the node, call the schema introspection endpoint (`GET /api/v1/ontology/schema/rules?domain=…`). Steps without an explicit `id` get a synthesized `${actionId}::${stepName}` so callers always have a stable handle.

Errors:

| HTTP | `error` |
|---|---|
| 400 | `missing-domain` |

| HTTP | `error` |
|---|---|
| 401 | `unauthorized` |
| 404 | `action-not-found` |
| 500 | `server-misconfigured` |
| 502 | `neo4j-unavailable` |

### `GET /api/v1/ontology/actions/{ref}/steps`

Returns the Action together with its `(:ActionStep)` children, ordered by `step.order`. Same lookup and domain rules as `/rules`, but the response omits the per-step rule list — useful when you only need the step structure (e.g. rendering an Action's pipeline).

Query params:

| Name | Required | Notes |
|---|---|---|
| `domain` | yes | Scopes both the Action match and the steps list. |

```
curl -sS \
  -H "Authorization: Bearer $ONTOLOGY_API_TOKEN" \
  "http://localhost:3500/api/v1/ontology/actions/matchResume/steps?domain=RAAS-v1" | jq
```

Response — `200 OK`: a single Action object with `action_steps[]` (no `rules` nesting):

```
{
  "id":          "matchResume",
  "name":        "matchResume",
  "description": "...",
  "action_steps": [
    { "id": "step-1", "name": "Hard requirements", "order": 1 },
    { "id": "step-2", "name": "Conflict-of-interest checks", "order": 2 }
  ]
}
```

Errors: same set as `/rules`.

### `POST /api/v1/ontology/actions/matchResume/results`

Persists a new `(:Candidate_Match_Result)` node, MERGE-ing `(:Candidate)` and `(:Job_Requisition)` stub nodes if they don't yet exist, then linking:

```
(:Candidate_Match_Result)-[:candidate_match_result_refers_to_candidate]->(:Candidate)
(:Candidate_Match_Result)-[:candidate_match_result_refers_to_job_requisition]->(:Job_Requisition)
```

Every call adds a **new** history record — the endpoint never overwrites.

Body:

```
{
  "candidateId":   "C-100023",
  "jobPositionId": "JR-50087",
  "result":        "匹配",
  "reason":        "…"
}
```

All four fields are required strings.

Response — `200 OK`:

```
{
  "candidateMatchResultId": "f3d1c8e2-2a4b-4d8c-9f1d-7e6c5b4a3f2d",
  "createdAt":              "2026-04-29T13:42:11.728+08:00"
}
```

`candidateMatchResultId` is a server-generated UUID v4 stored as `Candidate_Match_Result.candidate_match_result_id`. `createdAt` is the node's `datetime()` rendered as ISO 8601 with offset.

> **Why per-action and not generic?** Composite writes that MERGE foreign-key stubs (here: `Candidate`, `Job_Requisition`) and auto-link them are domain-shaped — the endpoint knows which input field maps to which target node and which relationship type to create. With a sample size of one, a generic `POST /instances/{label}` writer would have to encode that mapping in the request body and risks the wrong abstraction. Plan: keep per-action result writers for now; revisit a generic instance-writer once a second action (e.g. `scoreCandidate`, `generateOffer`) needs one and the shapes can be compared side-by-side.

---

## 5. Instance data CRUD

Endpoints under `/api/v1/ontology/instances/{label}/...` operate on **instance-data nodes** (records typed by a `:DataObject` schema), as opposed to §1 which operates on the schema definitions themselves. For example, `:DataObject {id: "Candidate", primary_key: "candidate_id"}` defines the schema (managed by §1), while `:Candidate {candidate_id: "C-100023", domainId: "RAAS-v1", name: "张三"}` is one instance row (managed here).

`{label}` is the schema's `id` (e.g. `Candidate`, `Job_Requisition`, `Candidate_Match_Result`). Every request:

- Looks up `:DataObject {id: $label, domainId: $domain}` to discover the **PK field name** via its `primary_key` property. Cached in- process for ~30s; first call to a `(label, domain)` pair pays a one-shot read of the schema.
- Returns `404 schema-not-found` if the schema doesn't exist in the domain. Distinct from `instance-not-found` (the schema is fine, but no instance row matches the supplied PK).
- Requires `?domain=…` like the rest of the API. Instance data is **domain-partitioned**: a `(candidate_id, RAAS-v1)` row is a different node from `(candidate_id, other-domain)`.
- Default mode is property-bag-permissive (any properties accepted, no field-level type checking) — same posture as §1. `?validate=strict` opts in to schema validation against the `:DataObject` definition.

The PK field is interpolated into Cypher (regex-gated by `safeProp`); the value is parameterized.

### `GET /api/v1/ontology/instances/{label}?domain=…` — list

```
GET /api/v1/ontology/instances/Candidate?domain=RAAS-v1&limit=50&candidate_status=active HTTP/1.1
Authorization: Bearer <token>
```

Response — `200 OK`:


```
{
  "items": [
    { "candidate_id": "C-100023", "domainId": "RAAS-v1", "name": "张三", "candidate_status": "active" },
    { "candidate_id": "C-100024", "domainId": "RAAS-v1", "name": "李四", "candidate_status": "active" }
  ],
  "nextCursor": null
}
```

Top-level filter params become Cypher equality predicates (same as §1). Reserved keys: `domain`, `limit`, `cursor`, `validate` — anything else is forwarded as `WHERE n.<key> = $<key>`.

### `POST /api/v1/ontology/instances/{label}` — create or bulk upsert

Single:

```
POST /api/v1/ontology/instances/Candidate HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId":     "RAAS-v1",
  "candidate_id": "C-100023",
  "name":         "张三",
  "age":          29
}
```

Bulk:

```
{
  "domainId": "RAAS-v1",
  "items": [
    { "candidate_id": "C-100023", "name": "张三" },
    { "candidate_id": "C-100024", "name": "李四" }
  ]
}
```

Semantics: `MERGE (n:{Label} {<pk_field>: $pkValue, domainId: $domain}) SET n = item`. Each item's PK comes from the body or — when absent — is server-generated via `randomUUID()`.

Response — `200 OK`:

```
{ "upserted": ["C-100023", "C-100024"], "count": 2, "generated": [] }
```

`upserted[]` carries the **final** PK of every row written, in input order. `generated[]` is the subset whose value the server filled in (callers can correlate position-by-position with `items[]`). When every item had a caller-supplied PK, `generated` is empty.

**Idempotency:** when every item has an explicit PK, retrying the same body produces the same graph state (`MERGE` is naturally idempotent). When the server generates IDs, retry is **not** idempotent — a network glitch + retry creates a second row. Callers that need retry-safety must supply PKs (or use `PUT` for known

IDs).

### `GET /api/v1/ontology/instances/{label}/{value}?domain=…` — read one

```
GET /api/v1/ontology/instances/Candidate/C-100023?domain=RAAS-v1 HTTP/1.1
Authorization: Bearer <token>
```

Response — `200 OK`: the property bag.

```
{
  "candidate_id":     "C-100023",
  "domainId":         "RAAS-v1",
  "name":             "张三",
  "candidate_status": "active",
  "age":              29,
  "updatedAt":        "2026-05-08T10:00:00.000+08:00"
}
```

`404 instance-not-found` if no node matches `(value, domainId)` on the schema's PK field. **No name-fallback** — instance data has no parallel "human-readable name" convention; the PK value is the only handle.

### `PUT /api/v1/ontology/instances/{label}/{value}` — replace

```
PUT /api/v1/ontology/instances/Candidate/C-100023 HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
  "domainId":         "RAAS-v1",
  "name":             "张三 (updated)",
  "candidate_status": "interviewing"
}
```

`MERGE (n:{Label} {<pk_field>: $value, domainId: $domain}) SET n = $props` — replaces all properties (the URL `{value}` and body `domainId` are merged in automatically). Properties absent from the body are removed. Idempotent. PUT on a non-existent row is functionally an upsert and returns `200`.

### `PATCH /api/v1/ontology/instances/{label}/{value}` — partial update

```
PATCH /api/v1/ontology/instances/Candidate/C-100023 HTTP/1.1
Content-Type: application/json
```

```
Authorization: Bearer <token>

{ "domainId": "RAAS-v1", "candidate_status": "offer-extended" }
```

Merges supplied keys; leaves others untouched. Set a key to `null` to remove a property. `404 instance-not-found` if no node matches.

### `DELETE /api/v1/ontology/instances/{label}/{value}?domain=…`

`DETACH DELETE` semantics: the node and all relationships touching it are removed. `404 instance-not-found` if no match.

```
{ "deleted": 1 }
```

## Validation

**Property-name validation runs on every write — default mode included.** The schema's `properties[]` declaration is treated as the contract for the legal field set. A POST/PUT/PATCH body containing a key that isn't in `properties[]` (or the PK or `domainId`) is rejected with `400 validation-failed` and `details.unknown: [<field-names>]`. This is true whether or not `?validate=strict` is set; strict mode only adds required-field and type checks on top.

For example, given a `:DataObject` Candidate schema with `properties: [ {name: "candidate_id"}, {name: "name"}, {name: "gender"}, {name: "mobile"}, {name: "email"} ]`, posting `{candidate_id: "C-1", name: "张三", age: 29}` returns 400 because `age` isn't declared. To accept `age`, add it to the schema first via §1's PATCH on `:DataObject`.

### `?validate=strict`

Available on `POST` / `PUT` / `PATCH`. When set, the request body is checked against the `:DataObject` schema's `properties[]` declaration:

- **Required fields** — every property with `is_required: true` must be present (POST/PUT) or non-null (PATCH).
- **Type checks** — scalar types (`string`, `integer`, `float`, `boolean`, `date`, `timestamp`) checked against the JSON value's type. List types (`List<string>`) checked for array shape.
- **Unknown fields** — always rejected with `400 validation-failed`, regardless of `?validate=strict`. The schema's `properties[]` is the contract: any field on the body that isn't declared in the schema is invalid.

Failure response — `400 Bad Request`:

```
{
  "error": "validation-failed",
  "message": "Required field `candidate_id` missing; field `age` expects integer, got string",
  "details": {
    "missing": ["candidate_id"],
    "type_errors": [{ "field": "age", "expected": "integer", "got": "string" }]
  }
}
```

On success — `200 OK` with the standard response (no `warnings` block). The `warnings.unknown_fields` field that earlier revisions emitted is no longer used: unknown fields are blocking, not advisory.

### Errors

| HTTP | `error` |
|---|---|
| 400 | `missing-domain` — `?domain=` absent |
| 400 | `invalid-json` — request body parse fail |
| 400 | `unsafe-label` — `{label}` path segment fails the regex gate |
| 400 | `validation-failed` — `?validate=strict` and the body has missing-required / type errors |
| 401 | `unauthorized` |
| 404 | `schema-not-found` — `:DataObject` doesn't exist for `(label, domain)`. Caller must define the schema first |
| 404 | `instance-not-found` — schema is fine, but no instance node matches `{value}` |
| 500 | `server-misconfigured` |
| 502 | `neo4j-unavailable` |

---

## Curl recipes

```
export TOKEN=$ONTOLOGY_API_TOKEN
export BASE=http://localhost:3500/api/v1/ontology
```

```bash
# 1) List DataObjects in a domain
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/objects?domain=RAAS-v1" | jq

# 2) Discover the live property catalog for Rules
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/schema/rules?domain=RAAS-v1" | jq

# 3) Bulk-upsert two Events
curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
        "domainId": "RAAS-v1",
        "items": [
          { "id": "ev-1", "name": "candidateCreated", "description": "…" },
          { "id": "ev-2", "name": "candidateUpdated", "description": "…" }
        ]
      }' \
  "$BASE/events" | jq

# 4) Patch a single Rule's description
curl -sS -X PATCH -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"domainId":"RAAS-v1","businessBackgroundReason":"new background"}' \
  "$BASE/rules/1-1-1" | jq

# 5) Create a HAS_FIELD link between two DataObjects
curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
        "domainId": "RAAS-v1",
        "type":     "HAS_FIELD",
        "fromId":   "Job_Requisition",
        "toId":     "field_title",
        "confidence": 0.92
      }' \
  "$BASE/links" | jq

# 6) Delete a DataObject (DETACH DELETE — also removes its links)
curl -sS -X DELETE -H "Authorization: Bearer $TOKEN" \
  "$BASE/objects/Job_Requisition?domain=RAAS-v1" | jq

# 7) Read the rules attached to an Action (matchResume here)
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/actions/matchResume/rules?domain=RAAS-v1" | jq

# 8) Read just the steps of an Action (no rule nesting)
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/actions/matchResume/steps?domain=RAAS-v1" | jq
```

```
# 9) Read an Action by name (id-then-name fallback works on bare GET too)
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/actions/matchResume?domain=RAAS-v1" | jq

# 10) Bulk-upsert Candidate instances (PK supplied — idempotent)
curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
        "domainId": "RAAS-v1",
        "items": [
          { "candidate_id": "C-100023", "name": "张三",  "candidate_status": "active" },
          { "candidate_id": "C-100024", "name": "李四",  "candidate_status": "active" }
        ]
      }' \
  "$BASE/instances/Candidate" | jq

# 11) Create a Candidate instance and let the server pick the UUID
curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "domainId": "RAAS-v1", "name": "王五" }' \
  "$BASE/instances/Candidate" | jq
# Response: { "upserted": ["<uuid>"], "count": 1, "generated": ["<uuid>"] }

# 12) Read one Candidate instance with strict-validation enabled
curl -sS -H "Authorization: Bearer $TOKEN" \
  "$BASE/instances/Candidate/C-100023?domain=RAAS-v1" | jq

# 13) Strict-validation write that surfaces a type error
curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "domainId": "RAAS-v1", "candidate_id": "C-bad", "age": "twenty-nine" }' \
  "$BASE/instances/Candidate?validate=strict" | jq
# Response: 400 validation-failed with details.type_errors populated
```

---

## Server configuration

Set on the Studio host ( `apps/studio/.env.local` or via deployment environment):

| Variable | Required | Purpose |
|---|---|---|
| `ONTOLOGY_API_TOKEN` | yes | Shared bearer token for all `/api/v1/ontology/*` endpoints. No default — endpoints return `500` until set. |

| Variable | Required | Purpose |
|---|---|---|
| `NEO4J_URI` | yes | Bolt endpoint, e.g. `bolt://localhost:7687` |
| `NEO4J_USER` | yes | Account |
| `NEO4J_PASSWORD` | yes | Password |
| `NEO4J_DATABASE` | no | Multi-db routing target (default: `neo4j`) |

The dev cluster default in this repo is `bolt://10.100.0.70:7687` — set `NEO4J_URI` accordingly.

---

## Caveats and known limitations

### Schema-agnostic by default = no field-level validation on schema CRUD

The API trusts the client by default. Sending `{"id": "x", "domainId": "y", "randomTypo": 123}` will store `randomTypo` on the node. Use the schema endpoint (§2) to verify field names; consider adding a check in your client. The repo's TypeScript types in `@allmeta/ontology-core` are a good source of truth for what fields *should* exist on each artifact — but the API does not enforce them on schema CRUD (§1).

Instance CRUD (§5) supports opt-in `?validate=strict` against the `:DataObject` schema definition. Schema CRUD has no equivalent — there's no meta-schema for what a `:DataObject` "should" look like.

### Reads do not auto-inflate JSON-stringified properties

If you POST `{"fields": [{"name": "title"}]}` you'll get back `"fields": "[{\"name\":\"title\"}]"`. Parse on the client. A future `?inflate=true` knob may add automatic re-hydration.

### Domain isolation is a hard rail

There is no cross-domain query in v1. Every Cypher includes `WHERE n.domainId = $domain`. To join across domains, the consumer must call the API once per domain and merge client-side. (This is a deliberate choice — most current consumers want one-domain-at-a-time semantics and accidental cross-domain reads are the most common bug class in the existing per-builder deploy routes.)

### Link type is part of identity, can't be PATCHed

Once a relationship has a type, you can't change it via PATCH — Neo4j relationships are typed at creation. `DELETE` the link and create a new one with the new type.

### `DELETE` on missing resources returns `404`, not `200`

`DELETE /objects/foo?domain=…` returns `404 node-not-found` when the ref resolves to neither an id nor a name. Same shape as `GET`. (Up through 2026-05-07 this returned `200 {deleted: 0}` instead — see the 2026-05-08 Changelog entry for the rationale.) A `200 {deleted: 0}` response is now only possible in the TOCTOU window between the resolver step and the delete itself.

### No bulk DELETE in v1

Loop over single deletes, or call the underlying Neo4j directly for mass cleanup. A `?bulk=true&where=…` deletion endpoint may follow once a real consumer needs it.

### No transactions across endpoints

Each HTTP call is its own Neo4j transaction. There is no `begin/commit/rollback` API. Multi-step graph mutations either go inside one `POST` (via the bulk-upsert body) or accept partial-failure risk between calls.

### Rate limits / retries

The dev cluster is small. Bulk-upsert in batches of ⩽500 items per call. There is no server-side rate limit in v1, but Neo4j will reject oversized parameter blobs (~1 MB) with `502`.

### What's not in v1

- No Workflow CRUD (intentionally — only the 5 listed kinds).
- No `ActionStep` as a top-level resource (surfaced only via `/actions/{ref}/rules` and `/actions/{ref}/steps`).
- No raw-Cypher passthrough endpoint.
- The per-app deploy routes in objects-builder / rules-builder / etc. are not part of this API; they keep using their own Neo4j drivers.
- No auth scopes / per-caller API keys — one shared token gates the whole surface.
- No request body schema validation against TypeScript types.

---

## Quick reference

```
# Generic node CRUD ({resource} ∈ objects, rules, actions, events)
# {ref} matches the node's id, then falls back to its name field
# (`name` for objects/actions/events; `businessLogicRuleName` for rules).
GET    /api/v1/ontology/{resource}?domain=…[&limit=&cursor=&id=&name=]
POST   /api/v1/ontology/{resource}                      body: {domainId, id, …} OR {domainId, items:[…]}
GET    /api/v1/ontology/{resource}/{ref}?domain=…
PUT    /api/v1/ontology/{resource}/{ref}                body: {domainId, …allProps}
PATCH  /api/v1/ontology/{resource}/{ref}                body: {domainId, …partial}
DELETE /api/v1/ontology/{resource}/{ref}?domain=…

# Schema introspection
GET    /api/v1/ontology/schema?domain=…
GET    /api/v1/ontology/schema/{resource}?domain=…[&sample=N]

# Links (relationships)
GET    /api/v1/ontology/links?domain=…[&type=&from=&to=]
POST   /api/v1/ontology/links                           body: {domainId, type, fromId, toId, …,
[linkId]}
GET    /api/v1/ontology/links/{linkId}?domain=…
PUT    /api/v1/ontology/links/{linkId}                  body: {domainId, …allProps}
PATCH  /api/v1/ontology/links/{linkId}                  body: {domainId, …partial}
DELETE /api/v1/ontology/links/{linkId}?domain=…

# Action sub-resources (composite — graph traversal / per-action writers)
GET    /api/v1/ontology/actions/{ref}/rules?domain=…            # single object: {...action,
action_steps:[{...step, rules:[…]}]}
GET    /api/v1/ontology/actions/{ref}/steps?domain=…            # single object: {...action,
action_steps:[…]} (no rule nesting)
POST   /api/v1/ontology/actions/matchResume/results             body: {candidateId, jobPositionId,
result, reason}

# Instance data CRUD ({label} = a :DataObject id; PK field discovered from schema)
# {value} = the PK value (e.g. candidate_id="C-100023"). No name-fallback on instances.
GET    /api/v1/ontology/instances/{label}?domain=…[&limit=&cursor=&<filter>=]
POST   /api/v1/ontology/instances/{label}                       body: {domainId, …props} OR {domainId,
items:[…]}
                                                                # PK omitted → server-generated UUID
(response.generated[])
GET    /api/v1/ontology/instances/{label}/{value}?domain=…
PUT    /api/v1/ontology/instances/{label}/{value}               body: {domainId, …allProps}
PATCH  /api/v1/ontology/instances/{label}/{value}               body: {domainId, …partial}
DELETE /api/v1/ontology/instances/{label}/{value}?domain=…
# Add ?validate=strict on POST/PUT/PATCH to opt into schema-aware
# required-field + scalar-type checks. Default mode is property-bag.

Auth (every endpoint):
```

```
Authorization: Bearer ${ONTOLOGY_API_TOKEN}
# or:  x-api-key: ${ONTOLOGY_API_TOKEN}
```

---

## Changelog

### 2026-05-08 — Always-on unknown-property rejection on instance writes

Tighten §5 instance CRUD: every POST/PUT/PATCH now validates body property names against the `:DataObject` schema's `properties[]` and rejects unknown keys with `400 validation-failed` — regardless of whether `?validate=strict` is set. The earlier `warnings.unknown_fields` soft-reporting on success was removed.

**Behavior change:**

| Mode | Before | After |
|---|---|---|
| Default (no `?validate=strict`) | Any property bag accepted; unknown keys silently stored | Unknown keys rejected with 400; `details.unknown: [<names>]` |
| `?validate=strict` | Required + type checks block; unknown keys reported on `warnings.unknown_fields` of the success response | Required + type + unknown checks all block; success response no longer carries `warnings` |

**Why:** the schema's `properties[]` declaration is the contract for what fields are legal on each `:DataObject`'s instances. Allowing arbitrary keys silently lets typos (`age` vs `user_age`) and hallucinated fields land in the graph undetected. ETL drift — historically the case-for-permissive — is better handled by updating the schema definition explicitly via §1's PATCH on `:DataObject` than by silent acceptance of new keys.

**Internal:** the schema-cache module now eagerly loads the `properties[]` catalog on first lookup (was lazy under the previous strict-only validation). Single Cypher query per cache fill instead of two.

**Migration:** producers must align their writes with the schema's `properties[]`. To accept a new field, PATCH the `:DataObject` schema first to declare it.

### 2026-05-08 — Instance data CRUD endpoints (`/api/v1/ontology/instances/{label}/...`)

A new top-level surface for **instance data** — Neo4j nodes typed by a `:DataObject` schema (e.g. `:Candidate`, `:Job_Requisition`, `:Candidate_Match_Result`). Parallels the existing schema-CRUD surface (§1) but operates on records rather than definitions.

**New endpoints (six):**

- `GET /instances/{label}?domain=…[&filter=]` — list with pagination + top-level equality filters.
- `POST /instances/{label}` — create one or bulk; PK comes from body or server-generated UUID.
- `GET /instances/{label}/{value}?domain=…` — read one by PK value.
- `PUT /instances/{label}/{value}` — replace all properties (idempotent upsert).
- `PATCH /instances/{label}/{value}` — partial merge.
- `DELETE /instances/{label}/{value}?domain=…` — `DETACH DELETE`.

**Key shape decisions:**

- Domain-partitioned: every endpoint requires `?domain=…`. A `(candidate_id, RAAS-v1)` row is a different node from `(candidate_id, other-domain)`.
- PK field discovered at runtime by reading `:DataObject.primary_key` for `(label, domain)`. Cached in-process for ~30s. `404 schema-not-found` if the schema doesn't exist; distinct from `instance-not-found`.
- No name-fallback on instances — the PK value is the only path segment. Bare CRUD (§1) still has id-then-alias-then-name resolution, but instance data has no parallel name convention.
- POST may omit the PK; the server fills in a UUID and reports it in `response.generated[]`. **POST loses idempotency on retry** when the server generates IDs — callers needing retry-safety supply PKs or use PUT.
- Single-column primary keys only in v1.
- Property-name validation runs on every write — unknown fields (not declared in the schema's `properties[]`) are rejected with `400 validation-failed` regardless of `?validate=strict`. Default mode is otherwise property-bag-permissive (consistent with §1): missing-required and type checks remain opt-in via `?validate=strict`. (See the same-day "Always-on unknown-property rejection on instance writes" entry above for the rationale.)

**Why a new surface (not a generalized §1):**

- §1 manages `:DataObject` / `:Rule` / `:Action` / `:Event` schema metadata — fixed set of four labels, slow rate of change. Instance data has open-ended labels (one per `:DataObject` schema) and high-frequency churn. Different rate-of-change patterns warrant different surfaces.

- §1 uses property-bag-permissive writes uniformly because there's no meta-schema to validate against. Instance writes have the `:DataObject` schema available and benefit from opt-in validation — the new surface is where that lives.
- Per-schema hand-coded endpoints (`POST /candidates`, `POST /job_requisitions`, …) don't scale; one generic surface dispatches by `{label}`.

**Migration note for `/actions/matchResume/results`:** The existing matchResume composite write MERGEs `:Candidate` and `:Job_Requisition` instance nodes **without** `domainId` — it predates the domain-partitioned model adopted here. Followup commit needed to align matchResume's MERGE Cypher with the new convention (or backfill `domainId` on existing rows). Tracked separately from the instance CRUD implementation.

### 2026-05-08 — Name-fallback on bare CRUD; `/actions/{ref}/steps`; `/rules` shape change

Three coordinated changes that align all `/actions/{ref}/...` endpoints on the same lookup + scoping convention, and add a steps-only read.

**Endpoint additions:**

- `GET /api/v1/ontology/actions/{ref}/steps?domain=…` — Action plus ordered `action_steps[]`, no per-step rule nesting.

**Endpoint behavior changes:**

*Bare CRUD (`/{resource}/{ref}`):*

| Endpoint | Before | After |
|---|---|---|
| `GET /{resource}/{id}`, `PUT/PATCH/DELETE /{resource}/{id}` | strict-id match | id matched first, then resource's name field (`name` for objects/actions/events; `businessLogicRuleName` for rules); id wins on ties. URL param renamed `{id}` → `{ref}` in docs but path shape is unchanged. |
| `DELETE /{resource}/{id}` on missing node | `200 OK { "deleted": 0 }` (idempotent silent success) | `404 node-not-found` (matches GET shape; surfaces typo'd refs to the caller). A `200 {deleted: 0}` response is now only possible in the TOCTOU window between the internal resolver step and the actual delete. |

*Action sub-resources (`/actions/{ref}/...`):*

| Endpoint | Before | After |
| --- | --- | --- |
| `GET /api/v1/ontology/actions/{id}/rules` | `?domainId=…` (optional, post-filter on rules; Action match was global) | `?domain=…` (required); Action match scoped by domain; steps + rules also filtered by domain. URL `{id}` becomes `{ref}` (id-then-name). |
| `GET /api/v1/ontology/actions/{id}/rules` response | array of one Action: `[{...action, action_steps: [...]}]` | single Action object: `{...action, action_steps: [...]}` |

**Why:**

- `/actions/{ref}/rules` was the only endpoint in the API where the base resource match was global (no domain scoping) and the param was spelled `?domainId=` instead of the `?domain=` used everywhere else. Aligning it pays the breaking-change tax once instead of twice.
- The array-of-one response shape on `/rules` was a historical artifact; `{ref}` always resolves to a single Action, so a single-object response matches the path semantics. `/steps` ships as single-object from day one to avoid carrying the same artifact forward.
- Bare CRUD callers regularly know an artifact only by name (e.g. `Job_Requisition` is the id, but `招聘岗位` is what humans use). Name-fallback removes the round-trip lookup callers had to do client- side, while keeping id matches deterministic on collisions.

**Migration:**

- Callers of `/actions/{id}/rules` must:
  - rename `?domainId=` → `?domain=` (param is now required);
  - replace `body[0]` indexing with direct field access on the response;
  - if relying on cross-domain matching, query each domain explicitly.
- Callers of `/{resource}/{id}` GET see no behavioral change unless they pass a name where they used to pass an id — that case now succeeds instead of 404'ing.
- Callers of `DELETE /{resource}/{id}` that depended on the old idempotent `200 {deleted: 0}` response on missing nodes must now expect `404 node-not-found`. If you were swallowing that response silently, you'll now see the 404 surface. (This is intentional — surfacing typo'd refs is more useful than silent success.)
- No back-compat shim. Internal call sites update in lock-step with this doc; downstream adopters switch over the same revision.

**Implementation notes:**

- Each bare-CRUD item operation now does two Cypher round-trips (resolver → main op). The resolver query is index-backed ( `(:Label, id, domainId)` ), so the latency delta is bounded — typically ~1ms on a hot index. Plan revisits this if the cost shows up in profiling.
- 404 messages on action sub-resources are standardized to `Action not found for selector "${ref}" in domain ${domain}` . The TOCTOU branch (an Action that disappeared between the resolver and the main query) returns a distinct `Action ${actionId} disappeared between resolve and fetch` so log-grep can tell the two cases apart. Error codes are unchanged ( `action-not-found` in both cases).
- The shared resolver lives at `handlers/nodes.ts` as `resolveIdFor(label, ref, domain): Promise<string | null>` , exported within the package and consumed by both `createNodeIdRoute` (bare CRUD) and the action sub-resource handlers. New action sub-resources should import it from there rather than reimplementing the id-then-name lookup.

**Post-rebase port (later same day):** when the branch was rebased onto a main that had independently expanded the legacy `match-resume` endpoint, the enhancements were merged forward into the new `/actions/{ref}/...` handlers rather than dropped. Specifically:

- `buildResolveRef` generalized from a single name-field to an ordered `matchFields` list. A new `ID_ALIAS_FIELDS: Record<NodeLabel, string[]>` config lets each label declare extra id-like columns; only `Action` uses it (carrying `action_id` between `id` and `name` in resolver priority).
- The `/actions/{ref}/rules` Cypher gained dual step discovery (linked via `HAS_STEP` plus fallback via `step.action_id` / `step.actionId` / `step.parentActionId` ), bidirectional rule traversal ( `(:Rule)-[:GOVERNS]->(:ActionStep)` AND `(:ActionStep)-[:CHECKS]->(:Rule)` ), and a `rules_json` / `rulesJson` / `rules` property fallback for rule ids not reachable via either relationship — enriched in a single batched follow-up query.
- Step ordering coalesces `s.order` → `s.index` → `"0"` . Steps without an explicit `id` are synthesized as `${actionId}::${stepName}` .
- The `/actions/{ref}/rules` response gained three derived fields: a top-level deduped `rules[]` , a `ruleCount` , and a compiled `userPrompt` (multi-line markdown ready to hand to an LLM).
- The `/actions/{ref}/steps` Cypher picked up the same dual step discovery and `coalesce` -based ordering. Its response shape is unchanged otherwise (single object, no rule nesting).

## 2026-05-07 — Action sub-resources replace `/match-resume/*`

The two purpose-specific endpoints under `/api/v1/ontology/match-resume/*` are renamed to live under `/api/v1/ontology/actions/{id}/...` so they compose with future Actions instead of forcing a new top-level namespace per action.

**Endpoint moves:**

| Before | After |
| --- | --- |
| `GET /api/v1/ontology/match-resume/rules?actionId=…&actionName=…&domainId=…` | `GET /api/v1/ontology/actions/{id}/rules?domainId=…` |
| `POST /api/v1/ontology/match-resume/result` | `POST /api/v1/ontology/actions/matchResume/results` |

**Behavior changes:**

- `actionId` / `actionName` query params on the rules read are folded into the path segment `{id}`, which is matched against `Action.id` first and falls back to `Action.name`. The implicit `actionName=matchResume` default no longer applies — the caller must name the action explicitly in the path. To migrate, replace `/match-resume/rules` (no params) with `/actions/matchResume/rules`.
- The result writer pluralizes `result` → `results` to match REST convention for create-on-collection (each call appends a new history record).
- Request body and response shapes are unchanged for both endpoints, so client code only updates URLs.

**Why:**

- The original `/match-resume/*` namespace was a temporary lift from rules-builder. As more actions need similar endpoints (e.g. `scoreCandidate`, `generateOffer`), the per-action sub-resource shape (`/actions/{id}/rules`, `/actions/{id}/results`) scales without adding a new top-level path each time.
- Filtering rules by an action is a graph traversal (`(:Action)-[:HAS_STEP]->(:ActionStep)<-[:GOVERNS]-(:Rule)`) which is meaningfully different from the property-equality filter on `GET /rules`. Keeping it on its own path (sub-resource of `/actions`) makes the distinction explicit instead of overloading `GET /rules` with a branch on `?actionId=`.
- A generic `POST /instances/{label}` writer was considered for the result endpoint but rejected at this stage. Composite writes that MERGE foreign-key stubs are domain-shaped (each action knows which inputs map to which target nodes / relationship types) and a one-sample generic body design risks the wrong abstraction. Keep per-action result writers for now; revisit when a second action needs one.

**No back-compat:** the old `/match-resume/*` paths are removed in this revision rather than aliased. Internal call sites update in lock-step with this doc; downstream adopters switch to the new paths. Search the repo for `match-resume` to find every site that needs updating.